Dedução do algoritmo de Bresenham

Marcelo R. Thielo

Introdução

O algoritmo de Bresenham foi criado para desenhar um segmento de reta entre dois pontos em um grid de pixels (raster) usando apenas aritmética de inteiros. Ele dispensa a necessidade de operações de ponto flutuante por meio do uso de um parâmetro de decisão que determina quais pixels situam-se mais próximos da linha ideal.

Considerações iniciais

Considere os pontos delimitadores do segmento:

$$(x_0, y_0) \quad \text{e} \quad (x_1, y_1)$$

e, assumindo que $x_1 > x_0$ e $y_1 > y_0$, definamos as dimensões de ambos os intervalos:

$$\Delta x = x_1 - x_0, \quad \Delta y = y_1 - y_0$$

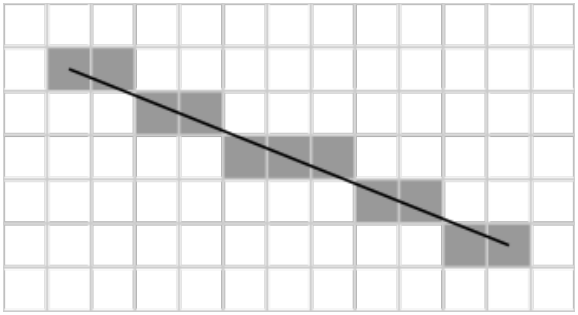


Figure 1: Linha desenhada com o algoritmo de Bresenham e linha ideal sobreposta

Para o primeiro octante, iremos assumir que $0 < a < 1$, de modo que a cada passo incrementaremos x por uma unidade e construiremos um critério para decidir quando devemos incrementar y.

A equação da reta

A equação da reta com inclinação a, que intercepta o eixo y, é dada por:

$$y = ax + b \quad \text{onde} \quad a = \frac{\Delta y}{\Delta x}$$

Dedução do parâmetro de decisão

A cada passo i, o pixel atual é denotado por (x_i, y_i). Dessa forma, precisamos escolher qual será o próximo pixel a ser "aceso".

Pixel adjacente à direita: $(x_i + 1, y_i)$

Pixel à direita, mas na linha imediatamente acima: $(x_i + 1, y_i + 1)$

Para tomar essa decisão, precisamos avaliar o pixel entre essas duas posições (aqui denotado M_i) que fica localizado em:

$$M_i = \left(x_i + 1, y_i + \frac{1}{2}\right)$$

Relembrando e aplicando a definição do coeficiente $a = \frac{\Delta y}{\Delta x}$ na equação da reta $y = ax + b$ e multiplicando ambos os lados por Δx, obtemos:

$$f(x, y) = \Delta y \cdot x - \Delta x \cdot y + b$$

(Utilizaremos o ponto matemático $(\cdot)$ para separar variáveis escalares que estão sendo multiplicadas, mas não confundir com produto escalar entre vetores.)

Dessa forma, aplicamos a função obtida sobre o ponto médio M_i, obtendo o parâmetro de decisão:

$$p_i = f(M_i) = \Delta y(x_i + 1) - \Delta x \left(y_i + \frac{1}{2}\right)$$

Esta função assumirá o valor zero sobre a linha, positivo acima, e negativo abaixo.

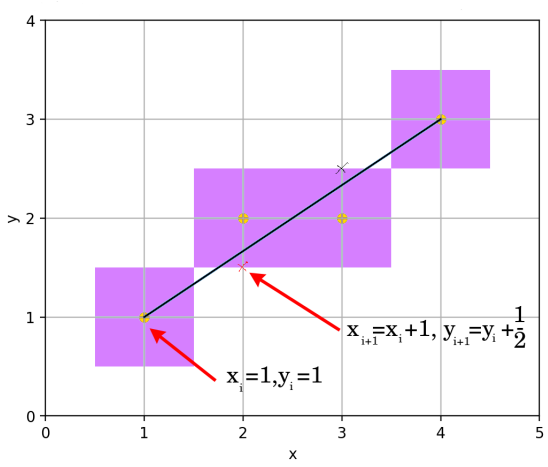


Figure 2: Posicionamento do ponto médio entre os dois candidatos (adjacente e diagonal)

Note que a fórmula ainda possui um termo denotado por uma fração. Como queremos eliminar divisões e necessidade de aritmética de ponto flutuante, esse termo pode ser facilmente eliminado, multiplicando ambos os lados da equação por 2:

$$\begin{aligned} p_i &= 2\Delta y(x_i + 1) - 2\Delta x \left(y_i + \frac{1}{2}\right) \\ &= 2\Delta y \cdot x_i + 2\Delta y - 2\Delta x \cdot y_i - \Delta x \end{aligned}$$

Reagrupando os termos:

$$p_i = (2\Delta y \cdot x_i - 2\Delta x \cdot y_i) + (2\Delta y - \Delta x)$$

O fator 2 do lado esquerdo pode ser descartado, pois apenas o sinal de p_i será relevante para o critério de decisão.

Valor inicial

No ponto inicial (x_0, y_0), uma vez que $x_0 = 0$ e $y_0 = 0$, o critério de decisão será inicializado com o valor:

$$p_0 = 2\Delta y - \Delta x$$

Atualizando p

Nesta parte iremos deduzir como p_i será atualizado de cada passo para o seguinte.

Caso 1: Pixel adjacente à direita

Caso $p_i < 0$, o próximo pixel desenhado será o imediatamente adjacente à direita:

$$(x_{i+1}, y_{i+1}) = (x_i + 1, y_i)$$

Substituindo na fórmula geral para p_i:

$$\begin{aligned} p_{i+1} &= (2\Delta y \cdot (x_i + 1) - 2\Delta x \cdot y_i) + (2\Delta y - \Delta x) \\ &= (2\Delta y \cdot x_i + 2\Delta y - 2\Delta x \cdot y_i) + (2\Delta y - \Delta x) \end{aligned}$$

Para obtermos Δp e atualizarmos p ao longo das iterações, subtraímos p_i de p_{i+1}, obtendo:

$$\Delta p = p_{i+1} - p_i = 2\Delta y$$

Assim, em um passo em que p_i satisfez a condição $p_i < 0$, atualizamos o valor do erro da seguinte maneira:

$$p_{i+1} = p_i + 2\Delta y$$

Caso 2: Pixel diagonal

Se $p_i \geq 0$, o próximo pixel é aquele imediatamente diagonal acima:

$$(x_{i+1}, y_{i+1}) = (x_i + 1, y_i + 1)$$

Substituindo o ponto acima na fórmula do critério p_{i+1} :

$$\begin{aligned} p_{i+1} &= (2\Delta y \cdot (x_i + 1) - 2\Delta x \cdot (y_i + 1)) + (2\Delta y - \Delta x) \\ &= (2\Delta y \cdot x_i + 2\Delta y - 2\Delta x \cdot y_i - 2\Delta x) + (2\Delta y - \Delta x) \end{aligned}$$

Subtraindo p_i para obter o Δp, ficamos com

$$\Delta p = p_{i+1} - p_i = 2\Delta y - 2\Delta x$$

Assim, em um passo em que condição $p_i \geq 0$ foi satisfeita, atualizamos o valor do erro da seguinte maneira:

$$p_{i+1} = p_i + 2\Delta y - 2\Delta x$$

Dessa maneira, concluímos a obtenção dos critérios para traçado de uma reta pelo algoritmo de Bresenham. Partindo da condição inicial $p_0 = 2\Delta y - \Delta x$, basta iterar:

$$p_{i+1} = \begin{cases} p_i + 2\Delta y & \text{if } p_i < 0 \quad \text{(Ao lado)} \\ p_i + 2\Delta y - 2\Delta x & \text{if } p_i \geq 0 \quad \text{(Diagonal para cima)} \end{cases}$$

Abaixo, uma versão em C do código para o primeiro octante. Lembre-se que os demais octantes e os casos em que $x_1 < x_0$ ou que $y_1 < y_0$ não estão cobertos nesta dedução. Deixamos esta tarefa para o astuto leitor.

```
/* Algoritmo de Bresenham em C (para o primeiro octante) */

void bresenhamLine(int x0, int y0, int x1, int y1) {
    int dx = x1 - x0;
    int dy = y1 - y0;

    int x = x0;
    int y = y0;

    int p = 2 * dy - dx;

    while (x <= x1) {
        putpixel(x, y);

        if (p < 0) {
            p = p + 2 * dy;
        } else {
            y = y + 1;
            p = p + 2 * dy - 2 * dx;
        }

        x = x + 1;
    }
}
```

Tarefas

1. Você seria capaz de produzir uma função capaz de desenhar uma linha, utilizando o algoritmo de Bresenham, independente dos valores de (x_0, y_0) e (x_1, y_1)? 2. Na função apresentada, ainda é possível fazer alguma otimização para melhorar o desempenho?